EFREI, M1 Big Data & Machine Learning

Module : API & Web Services

# Rapport de projet

Application de messagerie instantanée entre deux appareils


| | |
|---|---|
| **Étudiants** | KEITA Fode Laye |
| | BALDE Seyni Junior (ID20260097) |
| **Promotion** | ING2-APP-BDML1 |
| **Enseignant** | Ralph Bou Nader |


11 octobre 2026

# Table des matières

**1 Objectifs et mise en place** **2**

**2 Architecture** **2**

**3 API REST et authentification JWT** **3**

**4 Canal WebSocket** **4**

**5 Répartition REST et WebSocket** **5**

**6 Destinataire hors ligne** **6**

**7 Fonctionnalités bonus** **6**

**8 Clients mobile et bureau** **7**

**9 Tests et validation** **8**

**10 Difficultés rencontrées** **9**

**11 Tableau récapitulatif du scénario de démonstration** **9**

**12 Conclusion** **9**

**Annexe : fichiers joints** **9**

# 1 Objectifs et mise en place

Ce projet clôture le module en combinant les briques vues séparément : JSON (chapitre 3), REST (chapitre 4), WebSocket (chapitre 6) et sécurité par JWT (chapitre 7). Il s'agit d'une messagerie minimale mais complète : deux utilisateurs, l'un sur un téléphone et l'autre sur un ordinateur, échangent des messages en temps réel par l'intermédiaire d'un serveur central qui les enregistre.

Le serveur, ChatAPI, est écrit en Java 21 avec Spring Boot 4.1 (Web MVC, WebSocket, Security, Data JPA, Validation). Les données sont stockées dans une base H2 enregistrée sur disque, ou dans PostgreSQL avec le profil `postgres`. Les deux clients sont des pages HTML, CSS et JavaScript sans framework, servies par le serveur lui-même. Le projet se lance avec `gradlew bootRun`, sans autre prérequis que Java 17 : Gradle, le JDK du projet et la base sont fournis ou téléchargés automatiquement. Le code a été repris à partir d'une première base (entités, inscription et connexion), mise en conformité avec le sujet puis complétée.

# 2 Architecture

Les deux appareils ne communiquent jamais directement : chaque message passe par le serveur, qui l'enregistre puis le retransmet (figure 1). Chaque client utilise deux canaux : HTTP pour les opérations ponctuelles (compte, liste des utilisateurs, historique) et une connexion WebSocket permanente pour le temps réel.

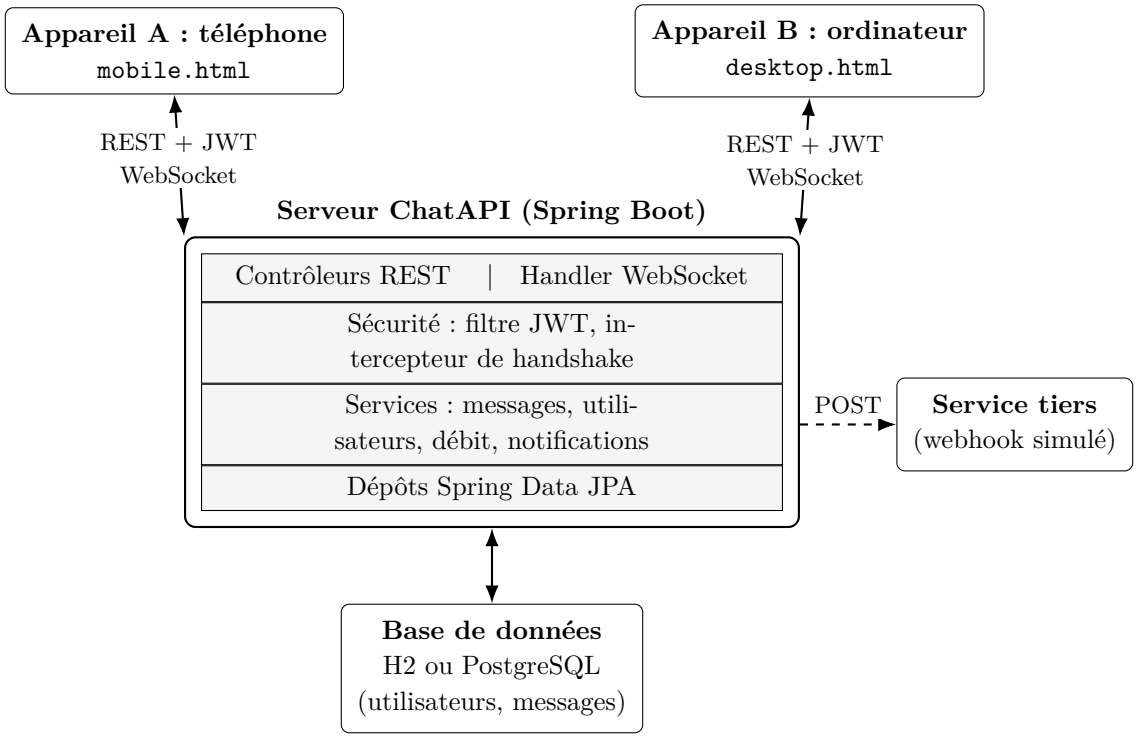



**Figure 1** – Architecture client-serveur : aucun échange direct entre les deux appareils.

Le serveur suit une architecture en couches : les contrôleurs et le handler WebSocket traitent les échanges, les services portent les règles métier, les dépôts accèdent à la base. Les objets JSON échangés sont des **DTO** (records Java) aux noms imposés par le sujet ; les entités JPA ne sont jamais exposées, ce qui garantit que le hachage du mot de passe ne quitte pas le serveur.

| Entité | Champs | Règles |
|---|---|---|
| Utilisateur | `id`, `username`, `password` | Nom unique en base ; mot de passe haché par BCrypt, jamais renvoyé. |
| Message | `id`, `expediteurId`, `destinataireId`, `contenu`, `dateEnvoi`, `statut` | Date fixée par le serveur ; statut `ENVOYE`, `LIVRE` ou `LU`. |

## 3 API REST et authentification JWT

| Méthode | URL | Rôle | Réponses |
|---|---|---|---|
| POST | `/api/comptes` | Créer un compte | 201, 400 invalide, 409 nom pris |
| POST | `/api/auth/login` | Recevoir un JWT | 200, 401 identifiants invalides |
| GET | `/api/utilisateurs` | Utilisateurs et statut `enLigne` | 200, 401 |
| GET | `/api/messages/{utilisateurId}` | Historique avec un utilisateur | 200, 401 |

La connexion renvoie un **JWT** signé en HMAC-SHA256. Le jeton se compose de trois parties encodées en Base64URL : l'en-tête, la charge utile (nom `sub`, identifiant `uid`, dates `iat` et `exp`, validité de deux heures) et la signature. La charge utile est lisible par tous ; seule la signature, que le serveur est le seul à pouvoir produire avec sa clé, garantit que le jeton n'a pas été modifié. La clé est lue dans la variable `JWT_SECRET`, jamais versionnée ; en son absence, une clé aléatoire est générée au démarrage.

Chaque requête protégée passe par le filtre `JwtAuthFilter`, qui lit l'en-tête `Authorization: Bearer` et vérifie signature et expiration. Le serveur ne garde aucune session : il est **sans état**, chaque requête porte sa propre preuve d'identité. Sans jeton valide, Spring Security répondrait `403` par défaut ; un point d'entrée dédié renvoie **401 Unauthorized**, comme l'exige le sujet (figures 2 et 3). L'utilisateur de l'historique est toujours celui du jeton : il est impossible de lire la conversation de deux autres personnes.




**Figure 2** – Swagger UI, `POST /api/auth/login` : 200 OK et jeton.

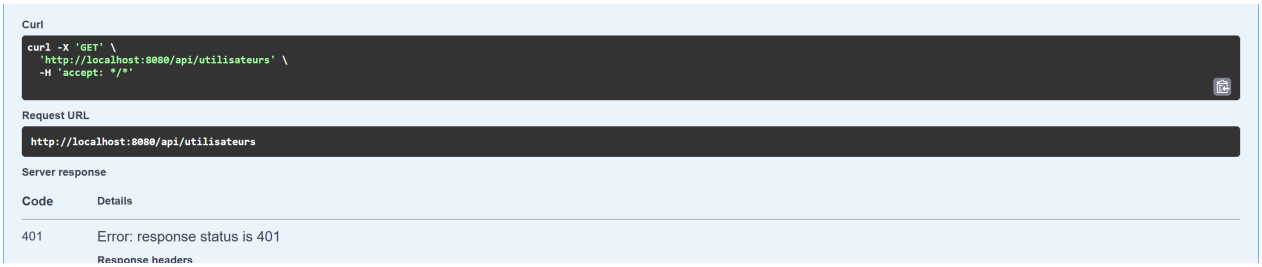



**Figure 3** – Swagger UI, `GET /api/utilisateurs` sans jeton : 401.

## 4 Canal WebSocket

Le client ouvre `ws://<hôte>:8080/ws/messages?token=<JWT>`. Le jeton est passé dans l'URL, car l'API WebSocket des navigateurs ne permet pas d'ajouter un en-tête `Authorization`. Il est vérifié une seule fois, à l'ouverture, par `JwtHandshakeInterceptor` : sans jeton valide, le handshake est refusé avec un 401 et le canal n'est jamais ouvert. L'identité est ensuite rattachée à la session pour toute sa durée.

Les connexions ouvertes sont tenues dans un registre (`SessionRegistry`) : une `ConcurrentHashMap` qui associe chaque utilisateur à ses sessions, car plusieurs threads la lisent et la modifient en même temps. Une session WebSocket n'accepte pas deux envois simultanés ; chacune est donc enveloppée dans un `ConcurrentWebSocketSessionDecorator`, qui met les envois en file.

| Type | Sens | Champs |
|---|---|---|
| `message` | client vers serveur | `destinataireId`, `contenu` |
| `message` | serveur vers les deux parties | `id`, `expediteurId`, `destinataireId`, `contenu`, `dateEnvoi`, `statut` |
| `accuse` | serveur vers expéditeur | `messageId`, `statut` |
| `lu` | client vers serveur | `interlocuteurId` |
| `saisie` | relayée au destinataire | `destinataireId` ou `expediteurId`, `enCours` |
| `presence` | serveur vers les connectés | `utilisateurId`, `enLigne` |
| `erreur` | serveur vers l'émetteur | `code`, `detail` |

La trame `message` reçue par le destinataire respecte le format imposé, complété de l'identifiant et du statut :

```
{"type": "message", "id": 12,
 "expediteurId": 1, "destinataireId": 2, "contenu": "Bonjour !",
 "dateEnvoi": "2026-10-11T21:40:12", "statut": "ENVOYE"}
```

Deux règles de sécurité s'appliquent à chaque message. L'expéditeur est celui du jeton vérifié au handshake, jamais celui du JSON reçu ; la date est celle de l'horloge du serveur. Un client qui envoie `"expediteurId":999` ou une date de l'an 2000 voit ces champs ignorés, ce que vérifie un test d'intégration. Le traitement suit ensuite toujours le même ordre : enregistrement en base, renvoi à l'expéditeur (qui obtient ainsi l'identifiant et la date du serveur), puis relais au destinataire s'il est connecté.

La figure 4 montre le scénario de démonstration : un message envoyé depuis l'ordinateur apparaît immédiatement sur le téléphone, sans action de l'utilisateur.

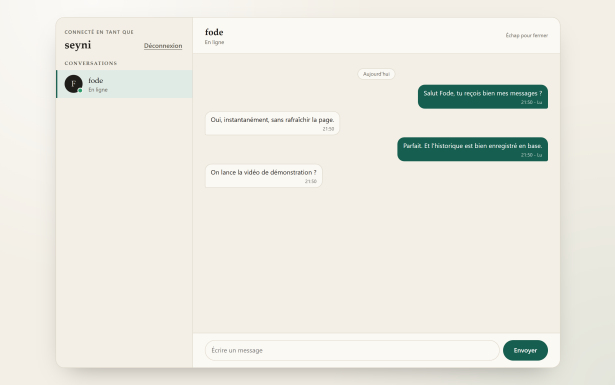



**Figure 4** – Client bureau (Seyni).




**Figure 5** – Client mobile (Fode) : message reçu.

## 5 Répartition REST et WebSocket

**Question à consigner : pourquoi exposer l'historique en REST plutôt qu'entièrement par WebSocket ?** Parce que l'historique est une **ressource** que l'on consulte ponctuellement, et non un flux d'événements. Il relève donc du modèle requête-réponse de REST :

- **Ressource identifiable par une URL** : `GET /api/messages/2` désigne sans ambiguïté la conversation avec l'utilisateur 2. On peut la tester dans Postman ou Swagger, la documenter, et la protéger avec les codes HTTP habituels (401).
- **Sans état** : la lecture ne dépend d'aucune connexion ouverte. C'est exactement ce qui est nécessaire à l'étape 5 du scénario : après une déconnexion, le client relit l'historique complet par une simple requête, avant même que le WebSocket soit rouvert.
- **Cache** : `GET` est une méthode sûre et idempotente, dont la réponse peut être mise en cache (en-têtes `Cache-Control`, `ETag`) ; un message envoyé sur un WebSocket ne bénéficie d'aucun de ces mécanismes.
- **Pagination et filtrage** : des paramètres de requête (`?avant=...&limite=50`) suffiraient pour charger une longue conversation par morceaux.

Faire passer l'historique par le WebSocket obligerait à réinventer ce que HTTP fournit déjà : associer chaque réponse à sa demande, signaler les erreurs, gérer l'authentification de chaque lecture. Le WebSocket est réservé à ce que REST ne sait pas faire : permettre au serveur de **pousser** une information sans requête préalable, avec une faible latence. La règle suivie dans tout le projet est donc : REST pour l'état (liste des utilisateurs, historique), WebSocket pour le changement d'état (nouveau message, accusé, présence, saisie). La liste des utilisateurs en est l'exemple le plus net : son état initial vient de `GET /api/utilisateurs` (champ `enLigne`), puis chaque connexion ou déconnexion est poussée par une trame `presence`.

## 6 Destinataire hors ligne

**Question à consigner : que faire si le destinataire n'est pas connecté ?** Le message ne doit pas être perdu : il est enregistré en base **avant toute tentative de livraison**, puis livré en différé. Le cycle de vie d'un message est le suivant :

1. enregistrement avec le statut `ENVOYE` ;
2. si le destinataire est connecté, relais immédiat, passage à `LIVRE` et accusé à l'expéditeur ;
3. sinon, le message attend en base et un webhook prévient un service tiers (section 7) ;
4. à la connexion du destinataire, ses messages en attente passent à `LIVRE` et leurs expéditeurs reçoivent un accusé ; il les lit dans l'historique REST ;
5. à l'ouverture de la conversation, le client envoie `lu` : les messages passent à `LU`.

Ce fonctionnement reprend le principe de **Kafka** vu au chapitre 6. Kafka ne supprime pas un événement parce qu'aucun consommateur n'est présent : il l'écrit dans un journal persistant, que chaque consommateur lit à son rythme à partir de sa position (offset). Ici, la table des messages joue le rôle du journal, et le statut `ENVOYE` celui d'un événement pas encore consommé. La persistance dissocie l'émission de la réception : l'expéditeur n'a pas besoin que le destinataire soit présent.

Les limites sont assumées pour un serveur unique. Le registre des connexions est en mémoire : avec plusieurs instances, il faudrait un intermédiaire partagé (Kafka, ou Redis en publication-abonnement) pour qu'une instance livre un message reçu par une autre. Le webhook n'est pas relancé en cas d'échec, puisque le message reste de toute façon disponible dans l'historique.

## 7 Fonctionnalités bonus

Les cinq fonctionnalités bonus proposées ont été réalisées.

| **Bonus** | **Mise en œuvre** |
|---|---|
| Accusés de réception | Trame `accuse` à chaque passage à `LIVRE` ou `LU` ; le client affiche Envoyé, Livré ou Lu sous chaque message envoyé. |
| Indicateur de saisie | Trame `saisie` relayée sans accès à la base ; une trame au début de la frappe, une après deux secondes d'inactivité. |
| Notification hors ligne | `POST` JSON vers `CHAT_WEBHOOK_URL`, exécuté en asynchrone (`@Async`) avec un délai maximal de deux secondes. Sans URL, le service tiers est simulé par la ligne de log « Notification envoyée à l'utilisateur… ». Le contenu du message n'est jamais transmis au tiers. |
| Limitation de débit | Fenêtre glissante en mémoire, au plus 20 messages par utilisateur sur toute période de 10 secondes ; au-delà, `erreur LIMITE_DEPASSEE` et message non enregistré. |
| OpenAPI et Swagger | springdoc-openapi ; documentation sur `/swagger-ui.html`, schéma `bearer` pour tester les endpoints protégés. |

**Statut qui ne recule pas.** Le relais au destinataire et la lecture s'exécutent dans deux threads différents. Si le destinataire lit le message avant que le thread de l'expéditeur ait enregistré `LIVRE`,

ce `LIVRE` tardif écraserait `LU`. La mise à jour est donc conditionnelle (`UPDATE ... WHERE statut IN (statuts précédents)`) et l'accusé n'est envoyé que si la ligne a réellement changé.

**Fenêtre glissante plutôt que fixe.** Une fenêtre remise à zéro chaque minute laisse passer deux rafales de part et d'autre de la remise à zéro. La fenêtre glissante conserve les dates des derniers envois et compte ceux des dix dernières secondes, quel que soit le moment.

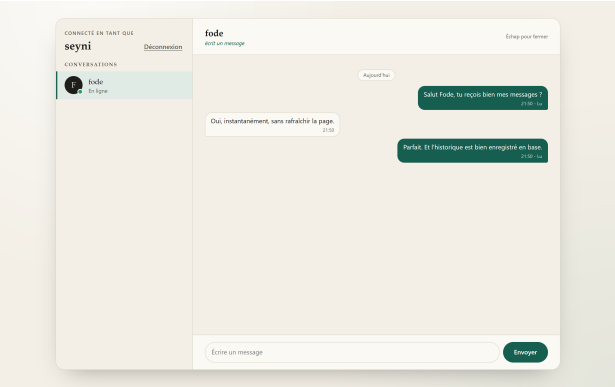



**Figure 6** – Statut Lu et indicateur de saisie.

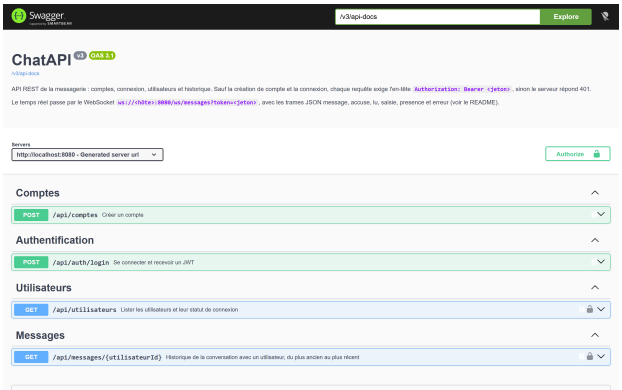



**Figure 7** – Documentation Swagger UI.

Sans `CHAT_WEBHOOK_URL`, un message envoyé à Fode déconnecté produit dans les logs du serveur la ligne suivante, écrite par un thread de l'exécuteur asynchrone (`task-1`) et non par celui du WebSocket :

```
INFO [task-1] NotificationService : Notification envoyée à l'utilisateur 2
     (service tiers simulé) : nouveau message de seyni
```

## 8 Clients mobile et bureau

Les deux clients sont des pages distinctes, `mobile.html` et `desktop.html`, qui partagent la même logique (`chat.js`) et le même thème ; seule la mise en page diffère. La version mobile affiche une vue à la fois (la conversation glisse par-dessus la liste des contacts), avec des zones tactiles d'au moins 44 pixels et des champs en 16 pixels pour éviter le zoom automatique d'iOS. La version bureau place contacts et conversation côte à côte. L'adresse racine redirige vers le client adapté à l'appareil. Tout texte reçu est inséré avec `textContent` et jamais `innerHTML` : un message contenant du HTML s'affiche tel quel au lieu d'être interprété (protection contre le XSS).

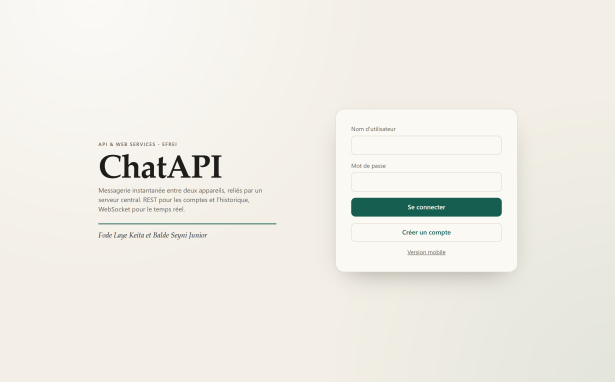



**Figure 8** – Écran d'accueil du client bureau.




**Figure 9** – Écran d'accueil du client mobile.

# 9 Tests et validation

Chaque étape a été validée par des tests automatisés, lancés par `gradlew test` sur une base H2 en mémoire.

| Classe | Tests | Ce qui est vérifié |
|---|---|---|
| `ApiRestTests` | 10 | Codes 201, 400, 409, 200 et 401 ; hachage BCrypt en base ; mot de passe jamais renvoyé ; historique identique des deux côtés et trié ; fichiers des clients et documentation accessibles sans jeton. |
| `WebSocketTests` | 9 | Handshake refusé sans jeton ; usurpation de l'expéditeur ignorée ; relais et accusés ; livraison à la connexion ; lecture ; présence ; saisie ; webhook ; limitation de débit ; trames invalides. |
| `RateLimiterTests` | 2 | Refus au-delà de la limite, glissement de la fenêtre, limites indépendantes par utilisateur. |
| `ChatapiApplicationTests` | 1 | Démarrage du contexte Spring. |

Les tests WebSocket démarrent un vrai serveur sur un port aléatoire et y connectent deux clients, Seyni et Fode. Le webhook est reçu par un petit serveur HTTP du JDK qui joue le rôle du service tiers.

Résultat de `gradlew test`, lancé depuis l'archive de rendu décompressée dans un dossier vierge, comme le ferait un correcteur :

```
BUILD SUCCESSFUL in 55s
ApiRestTests              tests=10  echecs=0  erreurs=0
ChatapiApplicationTests   tests=1   echecs=0  erreurs=0
RateLimiterTests          tests=2   echecs=0  erreurs=0
WebSocketTests            tests=9   echecs=0  erreurs=0
```

## 10 Difficultés rencontrées

| Problème | Cause | Solution |
|---|---|---|
| Gradle s'arrête faute de mémoire | Poste de 8 Go, démons Gradle accumulés | Mémoire du démon plafonnée dans `gradle.properties`, Java 21 |
| 403 au lieu de 401 sans jeton | Comportement par défaut de Spring Security | Point d'entrée `HttpStatusEntryPoint (UNAUTHORIZED)` |
| Authentifier le WebSocket | Aucun en-tête possible depuis un navigateur | Jeton dans l'URL, vérifié à l'interception du handshake |
| Envois concurrents sur une session | `sendMessage` n'est pas thread-safe | `ConcurrentWebSocket SessionDecorator` |
| `LIVRE` tardif écrasant `LU` | Relais et lecture dans deux threads | Mise à jour conditionnelle en base |
| Toutes les trames rejetées après l'ajout de `enCours` | Jackson 3 refuse un champ `boolean` absent | Type `Boolean`, valeur absente traitée comme `false` |
| Renommage des colonnes de `Message` | `ddl-auto=update` garde les anciennes colonnes obligatoires | Base de démonstration recréée ; un script de migration en production |

## 11 Tableau récapitulatif du scénario de démonstration

| Étape | Action | Échange | Résultat |
|---|---|---|---|
| 1 | Connexion de A (Seyni) | `POST /api/auth/login`, WebSocket | 200, jeton ; canal ouvert. |
| 2 | Connexion de B (Fode) | idem | A reçoit `presence`, Fode en ligne. |
| 3 | Message de A vers B | trame `message` | Enregistré, affiché sur B ; Livré puis Lu sur A. |
| 4 | Message de B vers A | trame `message` | Affiché immédiatement sur A. |
| 5 | Déconnexion puis reconnexion de A | `GET /api/messages/{id}` | Historique complet affiché. |

## 12 Conclusion

Ce projet assemble les notions du module en un système cohérent : REST porte les ressources (comptes, utilisateurs, historique) avec des URL, des méthodes et des codes standard, tandis que le WebSocket transporte ce qui doit être poussé sans attendre de requête. Le JWT relie les deux canaux : obtenu en REST, il authentifie chaque requête et l'ouverture du WebSocket, sans session côté serveur. La persistance systématique avant toute livraison, inspirée du journal de Kafka, garantit qu'aucun message n'est perdu, que le destinataire soit connecté ou non. Les principales difficultés ne venaient pas des API elles-mêmes mais de la concurrence entre threads et des changements de Spring Boot 4, que seuls les tests d'intégration ont permis de détecter.

## Annexe : fichiers joints

- `chatapi/` : code source du serveur et des deux clients (`src/main/resources/static/`) ;

- `README.md` : installation, lancement, configuration et accès depuis un téléphone ;
- `postman/` : collection Postman documentée de l'API REST ;
- ce rapport et la vidéo de démonstration.